

GROUP PROJECT

- Module: **CST2550 Software Engineering Management and Development**

Pseudocode: -

Postal Service Management System

Algorithm 1 — Add Parcel to Hash Table

Purpose:

- To add a new parcel into the Hash Table using its TrackingID as the key.

Pseudocode:

```
-> FUNCTION AddParcel(parcel):  
    index = HashFunction(parcel.TrackingID)  
    IF bucket[index] is empty THEN  
        bucket[index] = parcel  
    ELSE  
        ADD parcel to linked list at bucket[index]  
    END IF  
  
-> END FUNCTION
```

- Explanation:

- The HashFunction converts the TrackingID into an index number. The parcel is then stored at that index in the bucket array. If two parcels hash to the same index (collision), they are stored in a linked list at that bucket.

Algorithm 2 — Search Parcel in Hash Table

Purpose:

- To find a parcel in the Hash Table using its TrackingID.

Pseudocode:

```
-> FUNCTION SearchParcel(trackingID):  
    index = HashFunction(trackingID)  
    current = bucket[index]  
    WHILE current is not null DO  
IF current.TrackingID == trackingID THEN  
        RETURN current  
    END IF  
        current = current.next  
    END WHILE  
RETURN null  
  
-> END FUNCTION
```

- Explanation:

- The TrackingID is hashed to find the correct bucket. The linked list at that bucket is then traversed until the matching parcel is found. If not found, null is returned.

Algorithm 3 — Delete Parcel from Hash Table

Purpose:

- To remove a parcel from the Hash Table using its TrackingID.

Pseudocode:

```
-> FUNCTION DeleteParcel(trackingID):  
    index = HashFunction(trackingID)  
    IF bucket[index].TrackingID == trackingID THEN  
        bucket[index] = bucket[index]. Next  
    ELSE  
        TRAVERSE linked list at bucket[index]  
        REMOVE node where TrackingID matches  
    END IF  
-> END FUNCTION
```

- Explanation:

- The TrackingID is hashed to find the correct bucket. If the parcel is at the head of the linked list, it is removed directly. Otherwise, the list is traversed to find and remove the matching node.

Algorithm 4 — Insert Parcel into BST

Purpose:

- To insert a parcel into the Binary Search Tree in the correct sorted position by TrackingID.

Pseudocode:

```
-> FUNCTION BSTInsert(node, parcel):  
IF node is null THEN  
    RETURN new Node(parcel)  
END IF  
IF parcel.TrackingID < node.TrackingID THEN  
    node.left = BSTInsert(node.left, parcel)  
ELSE  
    node.right = BSTInsert(node.right, parcel)  
END IF  
RETURN node  
-> END FUNCTION
```

- Explanation:

- Starting at the root, the TrackingID is compared at each node. If smaller it goes left, if larger it goes right. This continues until an empty spot is found where the new parcel is inserted.

Algorithm 5 — Enqueue Delivery into Queue

Purpose:

- To add a new delivery to the back of the Queue.

Pseudocode:

```
-> FUNCTION Enqueue(delivery):  
    newNode = new Node(delivery)  
    IF queue is empty THEN  
        front = newNode  
        rear = newNode  
    ELSE  
        rear.next = newNode  
        rear = newNode  
    END IF  
-> END FUNCTION
```

- **Explanation:**
- A new node is created for the delivery. If the queue is empty both front and rear point to the new node. Otherwise, the new node is added to the rear of the queue. This follows First In First Out (FIFO) order.

Algorithm 6 — Dequeue Delivery from Queue

Purpose:

- To remove and return the delivery at the front of the Queue.

Pseudocode:

```
-> FUNCTION Dequeue ():  
    IF queue is empty THEN  
        RETURN null  
    END IF  
    temp = front  
    front = front.next  
    IF front is null THEN  
        rear = null  
    END IF  
    RETURN temp.data  
  
-> END FUNCTION
```

- Explanation:

- If the queue is empty null is returned. Otherwise, the front node is removed, and the next node becomes the new front. If the queue becomes empty after removal both front and rear are set to null.